WEEK 1 · LESSON 3 OF 24 · HANDOUT

Linear Regression with scikit-learn

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=7ArmBVF2dCs — open the lesson online to watch inside the course.

🎓 **Go deeper:** Data School: Data science in Python — pandas, seaborn and linear regression — https://www.youtube.com/watch?v=3ZWuPVWq7p4

🎓 **Go deeper:** freeCodeCamp: Machine Learning with Python and scikit-learn — full course — https://www.youtube.com/watch?v=hDKCxebp88A

Your goal in this lesson: Train a multiple linear regression model in scikit-learn, read its coefficients in plain words, and judge it with MAE, R^2 and a baseline.

From one feature to many

In Math & Python for AI you built linear regression from scratch with gradient descent. scikit-learn does the same job in one line — and with many features at once:

> $rent = w_1 \times size + w_2 \times distance + w_3 \times furnished + \ldots + b$

Each **weight** (coefficient) says how much the rent changes when that feature goes up by one, with the others held the same.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 600
district = rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], n, p=[0.45, 0.35, 0.2])
bedrooms = rng.integers(1, 5, n)
size_m2 = (bedrooms * 28 + rng.normal(20, 12, n)).clip(25).round()
km_to_centre = rng.uniform(0.5, 15, n).round(1)
furnished = rng.random(n) < 0.3
base = {"Gasabo": 120_000, "Kicukiro": 90_000, "Nyarugenge": 100_000}
rent = (np.array([base[d] for d in district]) + 2_600 * size_m2 - 6_000 * km_to_centre
        + np.where(furnished, 60_000, 0) + rng.normal(0, 25_000, n)).round(-3)
homes = pd.DataFrame({"district": district, "bedrooms": bedrooms, "size_m2": size_m2,
                      "km_to_centre": km_to_centre, "furnished": furnished, "rent_rwf": rent})
print(homes.shape)
homes.head()
```

Prepare and split

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

X = pd.get_dummies(homes.drop(columns="rent_rwf"), columns=["district"], dtype=int)
X["furnished"] = X["furnished"].astype(int)
y = homes["rent_rwf"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Train and read the model

```
model = LinearRegression().fit(X_train, y_train)

coefs = pd.Series(model.coef_, index=X.columns).round(0)
print(coefs)
print("intercept:", round(model.intercept_))
```

Read it like a sentence:

- **size_m2 ≈ +2,580**: each extra square metre adds about 2,580 RWF a month.
- **km_to_centre ≈ −6,400**: each kilometre further from town lowers rent by about 6,400 RWF.
- **furnished ≈ +59,000**: furniture adds about 59,000 RWF.
- **district_Gasabo** is about 30,000 RWF above **district_Kicukiro** — compare districts with each other, not with zero.
- **bedrooms ≈ +700** only: once size is known, the number of bedrooms adds almost nothing. Bedrooms and size carry the same information.

The model rediscovered the rule used to simulate the data (2,600 per m², −6,000 per km, +60,000 furnished). With real data you never know the true rule — that is why we test.

Evaluate honestly

```
pred = model.predict(X_test)
print("MAE:", round(mean_absolute_error(y_test, pred)), "RWF")
print("R²: ", round(r2_score(y_test, pred), 3))
```

You should get an MAE of about **20,000 RWF** and R² of about **0.94**. Using all features halved the error of the size-only model from Lesson 1 (about 42,000).

Always compare with a baseline

A model is only useful if it beats a lazy guess. The laziest regression guess is "every home costs the average rent":

```
from sklearn.dummy import DummyRegressor

base = DummyRegressor(strategy="mean").fit(X_train, y_train)
print("Baseline MAE:", round(mean_absolute_error(y_test, base.predict(X_test))))
```

The baseline is off by about **90,000 RWF**. Our model is more than four times better. Always report a baseline — it turns a number into a story.

Look at the mistakes

```
import matplotlib.pyplot as plt

plt.scatter(y_test, pred, alpha=0.6)
plt.plot([50_000, 600_000], [50_000, 600_000], color="red")   # perfect predictions
plt.xlabel("Real rent (RWF)")
plt.ylabel("Predicted rent (RWF)")
plt.title("Predicted vs real rent (test homes)")
plt.show()

errors = y_test - pred
print("Largest under-prediction:", round(errors.max()))
```

Points close to the red line are good predictions. Look for patterns: if all the big errors were furnished homes in Gasabo, the model would be missing something.

When linear regression struggles

- **Curved relationships** — it can only draw straight lines (unless you add features like size²).
- **Outliers** — one mistyped rent of 50,000,000 RWF pulls the whole line.
- **Features that move together** (size and bedrooms) — the coefficients become unstable and hard to read, even when predictions stay good. Regularisation (Lesson 16) helps.

Key terms

- **Multiple linear regression** — a straight-line model with several features.
- **Coefficient (weight)** — change in the prediction when one feature rises by 1.
- **Intercept** — the prediction when all features are 0.
- **R^2** — share of the variation explained (1 = perfect, 0 = no better than the mean).
- **Baseline** — a simple guess every model must beat.

Remember this: scikit-learn trains linear regression in one line; your job is to read the coefficients in plain words and prove the model beats a baseline on unseen data.

Name: _____ Date: _____

Activity

Task 1

Remove `bedrooms` from X and retrain. Does the test MAE change? What does that tell you?

Task 2

Predict the rent of a furnished 90 m² home in Kicukiro, 4 km from town. (Build a one-row DataFrame with the same columns as X.)

Task 3

Add a column `size_sq = size_m2 ** 2`. Does it help? Why or why not?

Task 4

Write three sentences for a landlord explaining what the model learned.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. The coefficient of km_to_centre is about −6,400. What does it mean?
   - A. Rent rises 6,400 RWF per km
   - B. The model is wrong
   - C. 6,400 homes are far from town
   - D. Each extra km from town lowers rent by about 6,400 RWF, other things equal
2. Why is the bedrooms coefficient so small?
   - A. It is a bug
   - B. Size already carries the same information
   - C. Bedrooms do not matter to anyone
   - D. Bedrooms were removed
3. What does $R^2 = 0.94$ mean?
   - A. The error is 94 RWF
   - B. 94 homes were used
   - C. 94% of predictions are exactly right
   - D. The model explains about 94% of the variation in rent
4. What does DummyRegressor(strategy='mean') predict?
   - A. The average training rent for every home
   - B. The best possible rent
   - C. Random rents
   - D. Zero
5. On a predicted-vs-real plot, where are perfect predictions?
   - A. On the diagonal line
   - B. On the left edge
   - C. At the bottom
   - D. Anywhere
6. Which situation is hard for plain linear regression?
   - A. Numbers on similar scales
   - B. A straight-line relationship
   - C. A strongly curved relationship
   - D. A clean dataset

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.